

// Jivops

Guía completa — 30 lecciones

GitHub Actions · Jenkins · Estrategias de despliegue · Seguridad ·
Métricas DORA

Contenido

Módulo 1: Fundamentos avanzados

- Lección 01 — Principios de CI/CD en producción
- Lección 02 — Pipeline as code — buenas prácticas
- Lección 03 — Trunk-based development y CI continuo
- Lección 04 — Feature flags y despliegue desacoplado de release
- Lección 05 — Gestión de artefactos (artifact repository)

Módulo 2: GitHub Actions avanzado

- Lección 06 — GitHub Actions — workflows avanzados
- Lección 07 — Reusable workflows y composite actions
- Lección 08 — Matrix builds en GitHub Actions
- Lección 09 — Environments y protection rules en GitHub Actions
- Lección 10 — Self-hosted runners

Módulo 3: Jenkins avanzado

- Lección 11 — Jenkins — Jenkinsfile declarativo avanzado
- Lección 12 — Jenkins shared libraries
- Lección 13 — Jenkins agents y distribución de carga
- Lección 14 — Jenkins pipelines paralelos
- Lección 15 — Migración de Jenkins a CI/CD moderno

Módulo 4: Estrategias de despliegue

- Lección 16 — Blue-Green deployments
- Lección 17 — Canary releases
- Lección 18 — Rolling deployments
- Lección 19 — Despliegues con feature flags avanzados
- Lección 20 — Rollback automatizado

Módulo 5: Seguridad en pipelines

- Lección 21 — Gestión de secretos en pipelines CI/CD
- Lección 22 — Escaneo de seguridad en el pipeline (SAST/DAST)
- Lección 23 — Firma y verificación de artefactos (supply chain security)
- Lección 24 — SBOM (Software Bill of Materials)
- Lección 25 — Least privilege en credenciales de CI/CD

Módulo 6: Observabilidad y madurez

Lección 26 — Métricas DORA — deployment frequency, lead time, MTTR, change failure rate

Lección 27 — Observabilidad de pipelines

Lección 28 — Optimización de tiempos de build

Lección 29 — Cultura de CI/CD y adopción en equipos

Lección 30 — Proyecto final: pipeline CI/CD completo de nivel élite (DORA)

Módulo 1: Fundamentos avanzados

Lección 01 — Principios de CI/CD en producción

CI valida cada cambio integrado con tests automatizados; CD automatiza llevarlo a producción. Integrar cambios pequeños y frecuentes reduce el riesgo frente a cambios grandes y espaciados.

Lección 02 — Pipeline as code — buenas prácticas

Definir el pipeline como código versionado permite revisarlo igual que cambios de código, con historial completo de quién modificó qué y cuándo.

Lección 03 — Trunk-based development y CI continuo

Integrar cambios pequeños y frecuentes a la rama principal, con ramas de feature de vida muy corta, reduce drásticamente los conflictos de merge grandes.

Lección 04 — Feature flags y despliegue desacoplado de release

Un feature flag permite desplegar código sin activarlo para los usuarios todavía, desacoplando el despliegue de la activación y reduciendo el riesgo del lanzamiento.

Lección 05 — Gestión de artefactos (artifact repository)

Un artifact repository almacena binarios versionados, listos para desplegar. Reconstruir en cada etapa arriesga que el binario probado no sea exactamente el que se despliega.

Módulo 2: GitHub Actions avanzado

Lección 06 — GitHub Actions — workflows avanzados

La sección 'on:' controla qué eventos disparan el workflow. Limitar el despliegue a producción solo a la rama main evita despliegues accidentales desde ramas de desarrollo.

```
on:
  push:
    branches: [main]
```

Lección 07 — Reusable workflows y composite actions

Un reusable workflow propaga un cambio a todos los repos que lo usan, sin duplicar mantenimiento entre decenas de repositorios de una organización.

Lección 08 — Matrix builds en GitHub Actions

Una estrategia matrix ejecuta el mismo job con múltiples versiones en paralelo, detectando problemas de compatibilidad que usuarios reales podrían tener.

```
strategy:
  matrix:
    node-version: [18, 20, 22]
```

Lección 09 — Environments y protection rules en GitHub Actions

Un environment con reglas de protección exige aprobación manual antes de desplegar, dando a una persona la oportunidad de confirmar antes de llegar a producción.

```
environment: production
```

Lección 10 — Self-hosted runners

Se usan cuando se necesita control total del entorno o acceso a redes internas privadas que los runners de GitHub en la nube pública no pueden alcanzar.

Módulo 3: Jenkins avanzado

Lección 11 — Jenkins — Jenkinsfile declarativo avanzado

La sintaxis declarativa sigue una estructura fija y predecible, reduciendo la curva de aprendizaje frente a la sintaxis scripted en Groovy libre.

```
pipeline {  
  agent any  
}
```

Lección 12 — Jenkins shared libraries

Una shared library centraliza lógica común de despliegue: un cambio se actualiza en un solo lugar y se propaga a todos los pipelines que la usan.

Lección 13 — Jenkins agents y distribución de carga

Múltiples agents distribuyen la carga de builds, evitando que todos compitan por el mismo recurso limitado del master y generen una cola lenta.

Lección 14 — Jenkins pipelines paralelos

El bloque 'parallel' ejecuta varias ramas de tareas independientes al mismo tiempo, como tests unitarios y linting, en vez de en secuencia.

Lección 15 — Migración de Jenkins a CI/CD moderno

Se recomienda auditar y documentar la lógica actual antes de migrar, y hacerlo proyecto por proyecto para limitar el impacto de cualquier problema.

Módulo 4: Estrategias de despliegue

Lección 16 — Blue-Green deployments

Se mantienen dos entornos completos idénticos; el tráfico cambia de golpe de uno a otro. El entorno anterior sigue disponible, permitiendo revertir casi al instante.

Lección 17 — Canary releases

Un pequeño porcentaje de tráfico ve la nueva versión antes de expandirla, limitando el impacto de un bug grave a solo esa fracción de usuarios.

Lección 18 — Rolling deployments

Las instancias se actualizan de a una o pocas a la vez, sin duplicar toda la infraestructura como en Blue-Green, aunque el rollback puede ser algo más lento.

```
maxUnavailable: 1
```

Lección 19 — Despliegues con feature flags avanzados

Un sistema de flags avanzado activa una función solo para un segmento específico de usuarios, validando el comportamiento real antes de exponer el riesgo a todos.

Lección 20 — Rollback automatizado

Un rollback automatizado se dispara cuando métricas de error o latencia superan un umbral, reaccionando de inmediato sin esperar a que una persona detecte el problema.

Módulo 5: Seguridad en pipelines

Lección 21 — Gestión de secretos en pipelines CI/CD

Las plataformas enmascaran automáticamente valores marcados como secretos, evitando que queden expuestos en los logs incluso al depurar por error.

Lección 22 — Escaneo de seguridad en el pipeline (SAST/DAST)

SAST analiza el código fuente estáticamente; DAST prueba la aplicación en ejecución. Combinar ambos da más cobertura que usar solo uno.

Lección 23 — Firma y verificación de artefactos (supply chain security)

Firmar un artefacto (con herramientas como cosign) protege que lo desplegado sea exactamente lo construido por el pipeline, sin alteraciones en el camino.

Lección 24 — SBOM (Software Bill of Materials)

Un SBOM es un inventario detallado de dependencias, permitiendo identificar rápido qué proyectos usan una librería vulnerable recién descubierta.

Lección 25 — Least privilege en credenciales de CI/CD

Limitar las credenciales de cada pipeline al entorno específico que despliega reduce el impacto de una dependencia comprometida a solo ese entorno.

Módulo 6: Observabilidad y madurez

Lección 26 — Métricas DORA — deployment frequency, lead time, MTTR, change failure rate

MTTR mide el tiempo promedio de recuperación ante un fallo. Equipos de alto rendimiento combinan despliegues frecuentes con MTTR bajo, porque cambios pequeños son más fáciles de aislar y revertir.

Lección 27 — Observabilidad de pipelines

Medir duración de cada etapa y tasa de fallos identifica exactamente dónde está el cuello de botella, permitiendo priorizar la optimización correcta.

Lección 28 — Optimización de tiempos de build

Paralelizar tests independientes y cachear dependencias reduce el tiempo total del pipeline, mejorando directamente la productividad diaria del equipo.

Lección 29 — Cultura de CI/CD y adopción en equipos

Arreglar un build roto de inmediato como prioridad es clave: si se ignora repetidamente, el pipeline deja de ser una señal confiable de que algo realmente falla.

Lección 30 — Proyecto final: pipeline CI/CD completo de nivel elite (DORA)

Se integra todo lo anterior: despliegues frecuentes y pequeños, lead time bajo, MTTR bajo, change failure rate bajo y rollback automatizado — el nivel esperado de un pipeline de elite según DORA.